HUDM 5001 Programming for Data Science
Fall 2026 Teachers College, Columbia University

Midterm Exam
ANSWER KEY

Tuesday, October 27, 2026
This exam covers everything through Week 7, including control structures and iterables.

Name: _____ **UNI:** _____

Time limit: 100 minutes.

Rules

1. Closed book. You may use **one** one-sided, letter-size sheet of notes (your "cheat sheet") and a calculator.
2. No computers, phones, tablets, smartwatches, earbuds, or AI tools. Put them away before the exam starts.
3. You will **not** write any code. Every question asks you to read, trace, explain, or choose.
4. Part A: choose one answer for each question and mark it on the answer grid on page 2. Only the grid is scored.
5. Part B: write your answers in the space provided. You may cite line numbers (for example, "line 12") instead of copying code. Show your reasoning, because partial credit is given.
6. Unless a question says otherwise, assume that `import json, os, sqlite3`, `import numpy as np` and `import pandas as pd` have already been run, that each snippet runs by itself in a fresh Python 3 session, and that no warning messages appear.

Part	Points	Score
A. Multiple choice (20 questions × 2.5 points)	50	
B1. Reading comprehension: academic research	25	
Total	**75**	

Do not turn the page until you are told to begin.

Designed by Chenguang Pan (cp3280@tc.columbia.edu), refined by Claude.ai.

Part A: Answer grid

Mark **exactly one** letter for each question. Only this grid is scored for Part A. If you change an answer, erase the old mark completely.

1.	A	**B**	C	D
2.	A	B	C	**D**
3.	**A**	B	C	D
4.	A	B	**C**	D
5.	A	**B**	C	D
6.	A	B	**C**	D
7.	**A**	B	C	D
8.	A	B	C	**D**
9.	A	**B**	C	D
10.	A	B	**C**	D

11.	A	**B**	C	D
12.	A	B	C	**D**
13.	A	B	**C**	D
14.	**A**	B	C	D
15.	A	**B**	C	D
16.	A	B	**C**	D
17.	A	B	C	**D**
18.	**A**	B	C	D
19.	A	B	C	**D**
20.	**A**	B	C	D

Part A: Multiple choice

20 questions × 2.5 points = 50 points. Choose the single best answer for each question and mark it on the answer grid (page 2).

1. What does this code print?

```
1 course = "HUDM"
2 code = course
3 course = course + " 5001"
4 print(code, "and", course)
```

A. `HUDM 5001 and HUDM 5001`
✓ **B.** `HUDM and HUDM 5001`
C. `HUDM 5001 and HUDM`
D. `HUDM and HUDM`

Key: B. `code = course` gives the string `"HUDM"` a second name. The next line builds a new string, `"HUDM 5001"`, and moves only the name `course` to it, so `code` still holds `"HUDM"`. *Tempting wrong answer:* A assumes `code` follows `course`. A string is never changed in place, so `course + " 5001"` makes a new string.

2. The Series `sizes` is built below. Which expression returns 25 **by its label**?

```
1 sizes = pd.Series([12, 30, 25, 8],
2                   index=["a", "b", "c", "d"])
```

A. `sizes.iloc["c"]`
B. `sizes.loc[2]`
C. `sizes.iloc[3]`
✓ **D.** `sizes.loc["c"]`

Key: D. `.loc` looks values up by label, and 25 sits under the label `"c"`. *Tempting wrong answer:* B mixes the two up: `.loc[2]` asks for a label 2, which does not exist, so it raises a `KeyError`. A raises a `TypeError` because `.iloc` only takes positions, and C is position 3, the value 8.

3. What does this code print?

```
1 df = pd.DataFrame({"id":    [1, 2, 3, 4],
2                    "score": [55, 82, 91, 67],
3                    "grade": ["F", "B", "A", "D"]})
4 print(df.shape, df["score"].shape, df[df["score"] > 60].shape)
```

Tip. Selecting one column with single brackets, `df["col"]`, gives a pandas Series, not a DataFrame.

✓ **A.** `(4, 3) (4,) (3, 3)`
B. `(3, 4) (4,) (3, 3)`
C. `(4, 3) (4, 1) (3, 3)`
D. `(4, 3) (4,) (3, 1)`

Key: A. The table has 4 rows and 3 columns. One column is a Series, which has only one dimension: `(4,)`. The mask keeps the three scores above 60 (82, 91 and 67) together with all 3 columns: `(3, 3)`. *Tempting wrong answer:* C gives the Series a second dimension. Only a DataFrame has columns.

4. What does this code print?

```
df = pd.DataFrame({"name":  ["Ana", "Ben", "Cai", "Dee", "Eli"],
                   "score": [72, 65, 88, 59, 91]})
conn = sqlite3.connect("class.db")
df.to_sql("roster", conn, index=False, if_exists="replace")
query = "SELECT COUNT(*) AS n FROM roster WHERE score > 70"
res = pd.read_sql_query(query, conn)
conn.close()
print(res.shape, res.loc[0, "n"])
```

A. `(5, 2) 3`
B. `(1, 1) 5`
✓ **C.** `(1, 1) 3`
D. `(3, 1) 3`

Key: C. The query returns one row with one column, `n`, so the result has shape `(1, 1)`. Three scores are above 70 (72, 88 and 91). *Tempting wrong answer:* B ignores the `WHERE` clause and counts all 5 rows. (D expects one result row per matching student; `COUNT(*)` collapses them into one number.)

5. What does this code print?

```
print(17 // 5, 17 % 5, -7 // 2)
```

A. `3 2 -3`
✓ **B.** `3 2 -4`
C. `3.4 2 -3.5`
D. `2 3 -4`

Key: B. `17 // 5` is 3, and `17 % 5` is the remainder, 2. `//` always rounds *down*, so $-7/2 = -3.5$ becomes -4. *Tempting wrong answer:* A rounds -3.5 toward zero. Rounding down means toward the smaller number, and -4 is smaller than -3.

6. Each line below is run by itself. Which one raises a `ValueError`?

A. `int("38")`
B. `int(3.8)`
✓ **C.** `int("3.8")`
D. `float("3.8")`

Key: C. `int()` can turn a string of digits into a whole number, but `"3.8"` has a decimal point, so it raises a `ValueError`. `float("3.8")` reads the decimal without trouble. *Tempting wrong answer:* B looks risky but works: `int(3.8)` drops the decimal part of a float and gives 3.

7. What does this code print?

```
lst = [1, 2, 3]
arr = np.array([1, 2, 3])
print(lst * 2, arr * 2)
```

✓ **A.** `[1, 2, 3, 1, 2, 3] [2 4 6]`
B. `[2, 4, 6] [2 4 6]`
C. `[1, 2, 3, 1, 2, 3] [1 2 3 1 2 3]`
D. `[2, 4, 6] [1 2 3 1 2 3]`

Key: A. Multiplying a list by 2 repeats the list. Multiplying a NumPy array by 2 multiplies every element, which is what vectorized means. *Tempting wrong answer:* B treats the list like an array.

8. What does this code print?

```
s = pd.Series([4, np.nan, 10, np.nan])
print(len(s), s.count(), s.mean())
```

Tip. `.count()` counts only the values that are not missing. It does not count NaN.

A. `4 4 3.5`
B. `2 2 7.0`
C. `4 2 3.5`
✓ **D.** `4 2 7.0`

Key: D. `len` counts all 4 entries, `.count()` counts the 2 that are not missing, and `.mean()` skips the missing values: $(4 + 10)/2 = 7.0$. *Tempting wrong answer:* C divides by all four entries ($14/4 = 3.5$), as if the missing values were zeros.

9. What does this code print?

```
votes = pd.Series(["yes", "no", "yes", "yes", "no"])
counts = votes.value_counts()
shares = votes.value_counts(normalize=True)
print(counts["yes"], shares["no"])
```

A. `3 2`
✓ **B.** `3 0.4`
C. `0.6 0.4`
D. `3 40.0`

Key: B. `value_counts()` counts each value, and `"yes"` appears 3 times. With `normalize=True` the counts become shares of the total: 2 of the 5 votes are `"no"`, so 0.4. *Tempting wrong answer:* D turns the share into a percent. `normalize=True` gives a proportion between 0 and 1.

10. What does this code print?

```
nums = [5, 2, 9]
a = sorted(nums)
b = nums.sort()
print(a, b, nums)
```

Tip. `a_list.sort()` sorts the list itself and returns None.

A. `[2, 5, 9] [2, 5, 9] [5, 2, 9]`
B. `[2, 5, 9] [2, 5, 9] [2, 5, 9]`
✓ **C.** `[2, 5, 9] None [2, 5, 9]`
D. `[2, 5, 9] None [5, 2, 9]`

Key: C. `sorted(nums)` returns a new sorted list. `nums.sort()` sorts `nums` itself and returns None, so `b` is None and `nums` is now `[2, 5, 9]`. *Tempting wrong answer:* D thinks `.sort()` leaves `nums` alone, the way `sorted()` does.

11. What does this code print?

```
1 df = pd.DataFrame({"id": [1, 1, 2, 3],
2                    "x":  [5, 5, 6, 7]})
3 df.drop_duplicates()
4 print(df.shape[0], df.drop_duplicates().shape[0])
```

Tip. `.drop_duplicates()` returns a new table. It does not change `df` itself.

A. `3 3`
✓ **B.** `4 3`
C. `4 4`
D. `3 4`

Key: B. `drop_duplicates()` returns a new table, and line 3 throws it away, so `df` still has 4 rows. The call inside `print` builds the de-duplicated table again, which has 3 rows. *Tempting wrong answer:* A expects line 3 to change `df` in place. (The fix would be `df = df.drop_duplicates()`.)

12. What does this code print?

```
1 n = 100
2 steps = 0
3 while n > 1:
4     n = n // 3
5     steps += 1
6 print(n, steps)
```

Tip. `//` rounds down, so `2 // 3` is 0.

A. `0 5`
B. `3 3`
C. `1 3`
✓ **D.** `1 4`

Key: D. The loop runs while `n > 1`: 100, then 33, 11, 3 and 1, which takes four passes. When `n` is 1 the condition is False, so the loop stops. *Tempting wrong answer:* C stops one pass too early. The condition is checked before each pass, and $3 > 1$ is still True, so the loop runs once more.

13. What does this code print?

```
1 df = pd.DataFrame({"q1": [4, 2, np.nan],
2                    "q2": [5, np.nan, 3]})
3 df["total"] = df["q1"] + df["q2"]
4 print(df["total"].sum(), df["total"].count())
```

Tip. A number plus a missing value is missing: 4 + NaN gives NaN.

A. `14.0 3`
B. `9.0 3`
✓ **C.** `9.0 1`
D. `nan 1`

Key: C. A missing value plus anything is missing, so `total` holds 9.0, NaN and NaN. `.sum()` skips the missing values and gives 9.0, and `.count()` counts the one value that is not missing. *Tempting wrong answer:* A treats the missing values as zeros.

14. What does this code print?

```
values = [4, -1, 6, 0, 9, 3]
total = 0
for v in values:
    if v < 0:
        continue
    if v == 0:
        break
    total += v
print(total)
```

✓ **A.** `10`
B. `22`
C. `9`
D. `21`

Key: A. `continue` skips -1, and `break` ends the loop when it reaches 0, so only 4 and 6 are added: 10. *Tempting wrong answer:* B ignores the `break` and keeps adding after the 0 ($4+6+9+3=22$).

15. What does this code print?

```
names  = ["Ana", "Ben", "Cai", "Dee"]
scores = [72, 58, 91, 65]
flags = []
for name, s in zip(names, scores):
    flags.append(name if s < 65 else "ok")
print(flags)
```

A. `['Ana', 'ok', 'Cai', 'Dee']`
✓ **B.** `['ok', 'Ben', 'ok', 'ok']`
C. `['ok', 'Ben', 'ok', 'Dee']`
D. `['ok', 58, 'ok', 'ok']`

Key: B. `zip` walks the two lists in step, so each name meets its own score. The one-line `if ... else` keeps the name only when the score is below 65, which happens only for Ben (58). Dee's 65 is not below 65, so Dee gets `"ok"`. *Tempting wrong answer:* C counts 65 as below 65. (A swaps the two branches, and D appends the score instead of the name.)

16. What does this code print?

```
a = pd.DataFrame({"score": [70, 80]})
b = pd.DataFrame({"score": [90, 60, 75]})
both = pd.concat([a, b])
print(both.shape, len(both.loc[0]), both.iloc[2]["score"])
```

Tip. Unless you set `ignore_index=True`, `pd.concat` keeps each table's original row labels (its index).

A. `(5, 1) 1 90`
B. `(5, 1) 2 75`
✓ **C.** `(5, 1) 2 90`
D. `(3, 2) 1 90`

Key: C. `pd.concat` stacks the 5 rows but keeps their old labels: 0, 1, 0, 1, 2. So `both.loc[0]` finds two rows (length 2), while `.iloc[2]` is the third row by position, which holds 90. *Tempting wrong answer:* A assumes the labels were renumbered 0 to 4. That only happens with `ignore_index=True`.

17. A SQLite table `grades` holds exactly the rows of `df` below. Which pandas expression gives the same names, in the same order, as `query`?

```
df = pd.DataFrame({"name":  ["Ann", "Ben", "Cy", "Dee", "Eve"],
                   "score": [85, 92, 78, 95, 60]})
query = """SELECT name FROM grades
           WHERE score >= 80
           ORDER BY score DESC"""
```

A. `list(df[df["score"] > 80].sort_values("score")["name"])`
B. `list(df.sort_values("score", ascending=False)["name"])`
C. `list(df[df["score"] >= 80]["name"])`
✓ **D.** `list(df[df["score"] >= 80].sort_values("score", ascending=False)["name"])`

Key: D. The query keeps the scores of at least 80 (Ann 85, Ben 92, Dee 95) and lists them from high to low: Dee, Ben, Ann. D uses the same mask and sorts in descending order. *Tempting wrong answer:* C has the right filter but keeps the original row order. (A sorts from low to high, and B forgets the `WHERE` filter.)

18. A SQLite table `orders` is shown below, and the cursor `cur` is connected to its database. What does the code print?

```
region | amount
-------+-------
east   | 40
west   | 70
east   | 60
north  | 30
west   | 45
north  | 20
```

```
query = """SELECT region, SUM(amount) FROM orders
           GROUP BY region ORDER BY region"""
rows = cur.execute(query).fetchall()
big = 0
for region, total in rows:
    if total >= 100:
        big += 1
print(len(rows), big, rows[0])
```

✓ **A.** `3 2 ('east', 100)`
B. `6 2 ('east', 40)`
C. `3 1 ('east', 100)`
D. `3 2 ('west', 115)`

Key: A. `GROUP BY` gives one row per region, in alphabetical order: east 100, north 50 and west 115. Two of the totals are at least 100 (east and west), and the first row is `('east', 100)`. *Tempting wrong answer:* C would be right for `total > 100`, which misses east's total of exactly 100.

19. What does this code print?

```
data = {"math": [80, 95, 70], "art": [70], "bio": [60, 90, 75]}
with open("marks.json", "w") as f:
    json.dump(data, f)
with open("marks.json", "r") as f:
    back = json.load(f)
best_subject = ""
best_n = 0
for subject, marks in back.items():
    if len(marks) > best_n:
        best_n = len(marks)
        best_subject = subject
print(best_subject, best_n, back == data)
```

A. `bio 3 True`
B. `math 3 False`
C. `art 1 True`
✓ **D.** `math 3 True`

Key: D. The JSON round trip gives back an equal dictionary, so `back == data` is True. The loop keeps the first subject with the most marks: math has 3, and bio's 3 is not *greater* than 3, so math stays. *Tempting wrong answer:* A replaces the leader on a tie. With `>` the tracker only changes for a strictly larger count.

20. What does this code print?

```
left = pd.DataFrame({"id":   [1, 2, 3, 4],
                     "math": [70, None, 85, 90]})
right = pd.DataFrame({"id":      [2, 3, 5],
                      "reading": [88, None, 75]})
m = pd.merge(left, right, on="id", how="outer")
report = {}
for col in m.columns:
    n = int(m[col].isna().sum())
    if n > 0:
        report[col] = n
print(report)
```

✓ **A.** `{'math': 2, 'reading': 3}`
B. `{'math': 1, 'reading': 1}`
C. `{'math': 1, 'reading': 3}`
D. `{'id': 0, 'math': 2, 'reading': 3}`

Key: A. The outer merge keeps ids 1 to 5. `math` is missing for ids 2 and 5, and `reading` for ids 1, 3 and 4. The `id` column has no gaps, so the `if` leaves it out of `report`. *Tempting wrong answer:* C is what a left join would give (id 5 dropped). B is the inner join, and D ignores the `if`.

Part B: Reading comprehension

One script, 25 points. Read the background, study the data and the script, then answer the questions that follow. Each question says which version of the code it is about. You never need to write code. Describe fixes in words and cite line numbers.

B1. [25 pts] Academic research: a math-anxiety follow-up

Setting. Jordan is a research assistant on a small study of a six-week mindfulness program that aims to lower math anxiety in ninth graders. Ten students filled out a math-anxiety questionnaire before the program started. Scores run from 20 to 60, and a higher score means more anxious. Five students were in the program (treatment) and five were in a comparison class (control). Two of them, one in each group, moved away before the post-test. Jordan wrote the script `anxiety_followup.py` (next page) to compare the two groups. The principal investigator (PI) gave Jordan these rules:

R1. Analyze only students who took both the pre-test and the post-test.

R2. A student's change is the pre-test score minus the post-test score. Anxiety going down is good, so a positive change means improvement.

R3. A change of 5 or more (≥ 5) counts as "improved", a change of -5 or lower (≤ -5) as "worse", and anything in between as "stable".

R4. For each group, report the number of students and the percent labeled "improved", rounded to 1 decimal.

R5. Report the gap in percent improved (treatment minus control), and save everything to `summary.json`.

The data. The two input files, exactly as pandas displays them after `pd.read_csv`:

`pre.csv`

	student_id	group	anxiety_pre
0	101	treatment	42
1	102	treatment	38
2	103	treatment	30
3	104	treatment	45
4	105	treatment	50
5	201	control	36
6	202	control	31
7	203	control	44
8	204	control	40
9	205	control	33

`post.csv`

	student_id	anxiety_post
0	101	34
1	102	33
2	103	30
3	105	38
4	201	34
5	202	36
6	203	38
7	205	32

Jordan's script: `anxiety_followup.py`

```python
# anxiety_followup.py: compare the change in math anxiety between two groups
import json
import pandas as pd

pre = pd.read_csv("pre.csv")      # student_id, group, anxiety_pre
post = pd.read_csv("post.csv")    # student_id, anxiety_post
print("pre:", pre.shape[0], "| post:", post.shape[0])

# Step 1: put each student's two scores side by side
df = pd.merge(pre, post, on="student_id", how="left")
print("rows after merge:", df.shape[0])

# Step 2: change = pre minus post (a positive change means less anxious)
df["change"] = df["anxiety_pre"] - df["anxiety_post"]

# Step 3: give every student a label
labels = []
for c in df["change"]:
    if c >= 5:
        labels.append("improved")
    elif c > -5:
        labels.append("stable")
    else:
        labels.append("worse")
df["label"] = labels

# Step 4: one summary line per group
summary = {}
for group in ["control", "Treatment"]:
    sub = df[df["group"] == group]
    n = sub.shape[0]
    n_improved = int((sub["label"] == "improved").sum())
    pct = round(n_improved / n * 100, 1)
    summary[group] = {"n": n, "improved": n_improved, "pct": pct}
    print(group, n, pct)

# Step 5: the gap between the groups, then save everything for the PI
gap = summary["treatment"]["pct"] - summary["control"]["pct"]
print("gap in % improved:", gap)
summary["gap"] = gap
with open("summary.json", "w") as f:
    json.dump(summary, f)
print("saved summary.json")
```

Terminal output. This is what the terminal shows when Jordan runs the original script:

```
$ python anxiety_followup.py
pre: 10 | post: 8
rows after merge: 10
control 5 20.0
Traceback (most recent call last):
  File "anxiety_followup.py", line 33, in <module>
    pct = round(n_improved / n * 100, 1)
                ~~~~~~~~~~~^~~
ZeroDivisionError: division by zero
```

About the script. It contains exactly one bug and one PI's rule violation. The bug stops the script with an error message. The rule violation does not stop anything, but it makes the numbers wrong. Each question below says which version of the code it is about.

(a) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Line 42 saves the dictionary `summary` to the file `summary.json`. Explain what the file contains: its keys, and what each value holds. You do not need to calculate any numbers.

Key:

Answer. The file holds one dictionary with three keys. `"control"` and `"treatment"` each map to a smaller dictionary with three keys: `"n"`, the number of students in that group; `"improved"`, how many of them are labeled "improved"; and `"pct"`, the percent labeled "improved", rounded to 1 decimal. `"gap"` holds a single number, the treatment percent minus the control percent. (For reference, with both problems fixed: control has n 4, improved 1 and pct 25.0; treatment has n 4, improved 3 and pct 75.0; the gap is 50.0.)

(b) **[4 pts]** *Code version: Original script (this answer does not depend on the bug or the rule violation).* In plain English, what does Step 3 (lines 16–25) do? Why does the order of the conditions on lines 19 and 21 matter?

Key:

Answer. Step 3 goes through the students' changes one at a time and adds one label per student to the list `labels`: "improved" if the change is 5 or more, "stable" if it is above -5, and "worse" otherwise. Line 25 stores the list as a new column. The order matters because Python runs only the first condition that is True. Line 19 checks `c >= 5` first, so line 21 only ever sees changes below 5. If the two conditions were swapped, a change of 8 would pass `c > -5` first and be labeled "stable", and nobody would ever be labeled "improved".

(c) **[6 pts]** *Code version: Original script.* Look at the terminal output above: the script prints three lines and then stops with an error. (i) Which line raises the error, and what does the message mean? (ii) Which other line is the real cause? Use the data and the printed lines as evidence. (iii) Describe in words how to fix it.

Key:

Answer. (i) Line 33. The message means the script divided by zero: `n` is 0, so `n_improved / n` cannot be computed. (ii) The real cause is line 29. The list spells the group `"Treatment"` with a capital T, but `pre.csv` says `"treatment"`. Python compares strings exactly, capital letters included, so the mask on line 30 keeps no rows and `n` on line 31 is 0. The terminal output agrees: the control line was printed, and the crash came on the second pass of the loop, the one for `"Treatment"`. (iii) Spell the group exactly as the data does: `"treatment"` on line 29.

(d) **[6 pts]** *Code version: Assume the error in (c) has been fixed, so the script runs to the end.* Even so, one line breaks one of the PI's rules, and Python does not give any error message. (i) Which line is it, and which rule does it break? (ii) Explain the mistake, including what happens to students 104 and 204. (iii) Explain how to revise that line.

Key:

Answer. (i) Line 10, which breaks R1. (ii) A left join keeps every student in `pre.csv`, including 104 and 204, who have no post-test; the terminal output shows `rows after merge: 10` although only 8 students took both tests. Their `anxiety_post` is NaN, so their change is NaN as well. In Step 3 both comparisons with NaN are False, so the `else` labels them "worse". Step 4 then counts them, so each group has 5 students instead of 4 and the percent improved comes out too low. (iii) Use `how="inner"` on line 10, so that only students who appear in both files stay. Dropping the rows with a missing post-test before Step 2 also works.

(e) **[5 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Rule R3 defines the three labels. Does Step 3 follow R3, including the boundary values 5 and -5? Answer Yes or No, then explain. Use students 102 and 202 in your answer: their changes are exactly 5 and -5.

Key:

Answer. Yes. Student 102 has a change of 5: `c >= 5` on line 19 is True, so 102 is "improved", as R3 says. Student 202 has a change of -5: `c >= 5` is False, and `c > -5` on line 21 is also False because -5 is not greater than -5, so 202 falls to the `else` and is "worse", again as R3 says. Every change strictly between -5 and 5 passes line 21 and is "stable".

End of exam